DSA Mini Project Report

Browser History Navigation System using Doubly Linked List (C++)

Roll No	Name
23359	Kalbhor Vivek Manoj
23362	Adgurwar Aaryan Prashant
23366	Patel Mayank
23367	Patil Jayesh Vinod

GitHub Repository: https://github.com/Aaryan200621/DSA

1. Aim

To design and implement a browser history system that supports visiting pages, moving back and forward, searching, deleting and saving history, using a **doubly linked list** in C++ with a simple web interface served by a custom C++ server.

2. Problem Statement

A browser must remember visited pages in order, let the user move to the previous or next page in constant time, discard forward pages when a new page is visited from the middle of history, and keep only a limited number of recent pages. An array needs costly shifting for insertions and deletions, so a dynamic structure with two-way links is required.

3. Data Structure Used

A **doubly linked list** where each node stores a URL with `prev` and `next` pointers. Three variables manage it: `head` (oldest page), `current` (page being viewed) and `count` (pages stored, capped at `LIMIT = 10`). `prev` gives Back and `next` gives Forward in O(1), and a node can be unlinked without searching for its predecessor.

```
struct Node {
    string url;
    Node* prev = nullptr;
    Node* next = nullptr;
};
```

4. Implementation Overview

The `BrowserHistory` class wraps the list. Core logic of adding a page:

```
void addPage(string url) {
    if (current == nullptr) head = current = new Node(url);
    else {
        deleteForwardHistory();          // clear pages ahead of current
        Node* n = new Node(url);
        current->next = n;  n->prev = current;  current = n;
    }
    count++;
    if (count > LIMIT) {                 // drop the oldest page
        Node* t = head;  head = head->next;
        head->prev = nullptr;  delete t;  count--;
    }
}
```

Operation	Method	Description	Time
Visit	visitPage	Insert after current, remove forward history	O(k), k = forward nodes
Back / Forward	goBack, goForward	Move current via prev / next	O(1)
Search	searchHistory	Linear scan for a URL, returns index	O(n)
Delete	deletePage	Unlink node at index, fix head / current	O(n)
Clear	clearHistory	Delete all nodes, empty the file	O(n)
Save / Load	saveHistory, loadHistory	Write / read URLs in history.txt	O(n)

Space complexity is O(n). Since history is limited to 10 pages, every operation is effectively very fast. The destructor frees all nodes, so there are no memory leaks.

5. Web Server Architecture

The program is also a small HTTP server built with POSIX sockets, listening on `localhost:8080` (loopback only). It serves front-end files from the `public` folder and exposes REST-style endpoints. Each reply is a JSON object with the message, current page, index, full history list and the `canBack` / `canForward` flags, so the interface can enable or disable buttons.

Endpoint	Action
/api/visit	Visit a new URL
/api/back, /api/forward	Navigate through history
/api/search, /api/delete	Find or remove an entry
/api/save, /api/load, /api/clear	File persistence and reset
/api/state	Get the current history state

Build and run: `g++ -std=c++17 server.cpp -o server`, then `./server` and open `http://localhost:8080`.

6. Sample Output

Visiting `google.com` then `youtube.com` and pressing Save creates `history.txt`:

```
google.com
youtube.com
```

Pressing Back moves to `google.com`. Visiting a new page from there erases `youtube.com` from forward history, just like a real browser.

7. Features

- Visit, Back and Forward navigation
- Automatic removal of forward history on a new visit
- History limit of 10 pages (oldest page is dropped)
- Search and delete a specific history entry
- Save, load and clear history using a file
- Input trimming and JSON escaping for safe replies

8. Conclusion

The project shows how a doubly linked list models real browser history efficiently, with constant-time navigation and simple pointer-based insertion and deletion. It also covers dynamic memory management, file handling and socket programming in C++. Future improvements: a hash map for O(1) search, timestamps per page, and multi-user support.

Source code: https://github.com/Aaryan200621/DSA